

Autonomous Multi-Agent Search and Rescue: Path Planning

Jack Millington¹, Jayden Knight¹, Callum Brown¹, Yusuf Abushaaban¹

¹ Griffith University, Southport QLD, Australia

Abstract. The use of Autonomous underwater vehicles (AUVs) are vital for modern search and rescue (SAR) operations, specifically in environments humans can't access. This report builds on a ROS-based aquatic SAR scenario by introducing a multi-agent system designed to improve speed, coordination and operational clarity. The proposed system consists of three agents. The first agent is a search AUV. This agent is responsible for locating all survivors. The second agent is a rescue AUV. This agent is tasked with retrieving all survivors detected by the search AUV. The final agent is a central Search and Rescue Manager agent. This agent coordinates the timing and task allocation between the two agents. By dividing the SAR phases and delegating responsibilities to agents, the system achieves efficient task execution and reduces delays in operation. Integration with ROS and PAT enables intelligent path planning, real-time decision making and flexible coordination. The effectiveness of this approach is evaluated through simulation, demonstration of improved responsiveness and overall performance in aquatic SAR environments.

Keywords: Artificial Intelligence Decision-making, ROS, PAT, Multiple Agent Search and Rescue

1 Introduction

Autonomous robotic systems have become a staple in modern search and rescue (SAR) operations and prove to be extremely effective in hazardous or inaccessible environments. These systems, such as unmanned aerial vehicles (UAVs) and autonomous underwater vehicles (AUVs), have enhanced the efficiency and safety of SAR missions with their mobility, sensor integration, and adaptability.

The integration of robust platforms like the Robot Operating System (ROS) with intelligent decision-making frameworks such as Process Analysis Toolkit (PAT), have enabled robots to perceive the environment, plan optimal paths, and dynamically adjust to new information. AUVs have demonstrated their effectiveness in SAR missions. Notably, the Bluefin-21 AUV was deployed in the search for Malaysia airlines flight 370, showcasing its capability to operate in deep sea conditions, and cover extensive areas effectively (Maxey, 2014) [1].

Our project extends a previously developed ROS-based aquatic SAR simulation, by introducing a multi-agent architecture. Specifically, we propose a triple agent system comprising two distinct AUVs, and a third agent controlling the two AUVs. One AUV is dedicated to comprehensively exploring and mapping the environment, while the other is focused on the most efficient retrieval of survivors. The third agent controls both of the controllers used for each AUV. This division of labour aims to optimise all operations, enhancing both coverage efficiency, and responsiveness in rescue operations.

Our approach maintains the core principle of integrating high-level intelligent planning, with low-level actuation and sensing in ROS. By distributing tasks between specialised agents, we aim to demonstrate improved teamwork and coordination, addressing key challenges in modern SAR robotics. The below sections of this report outline the related work in this field, detail the architecture of our multi agent implementation, and present an experimental evaluation of its performance.

2 Related work

Search and rescue (SAR) robotics has increasingly relied on autonomous systems capable of navigating dangerous or inaccessible terrain, mapping unknown environments, and making intelligent decisions in real time. The integration of high-level AI planning with low-level ROS-based control forms the basis of modern approaches. Our project builds on this foundation by dividing responsibilities between three agents: an Explorer AUV, a Rescuer AUV, and a Controller Agent that controls the two others. This modular approach improves scalability, decision clarity, and robustness.

The following studies are particularly relevant, each providing insight into planning, autonomy, and system integration.

2.1 Autonomous Search and Rescue: Path Planning

Todd et al. (2023) [2] explored the idea of creating a simulated space within ROS where hostiles and survivors would be randomly placed for an AUV to explore the area, locate all survivors, and then rescue all survivors. They integrated PAT with ROS to enable the AUV to use high-level decision-making for controlling have current action and route the AUV should be currently taking. This approach directly relates to the type of system that we intend to improve upon by updating the system to contain multiple agents. Given that this report provides all of the source code for their system, this single-agent system was chosen as the basis for creating our multi-agent system. Building off of an already existing system also means that comparing our multi-agent system and a single-agent system to see which is better is made far easier as we can use the original system as the testing counterpart to our system.

2.2 AUV for Search & Rescue at Sea: An Innovative Approach

Venkatesan (2016) [3] outlines the use of real-world AUVs for marine SAR operations, equipped with sensors like sonar and magnetometers. The paper highlights the advantages of AUVs such as autonomy, and safety, along with the ability to detect and report survivor locations in harsh underwater environments. In our simulation, these concepts are used directly through the Explorer AUV, which detects survivors, and the Rescuer AUV, which navigates to their positions to retrieve them after the entire area has been explored. Sensor modelling in the ROS-Gazebo framework mimics sonar and infrared detection to locate survivors. The communication of survivor locations from the explorer to the controller agent is similar to how real AUVs send their findings back to a base station.

2.3 GRAVITAS: A Model Checking Based Planning and Goal Reasoning Framework

Bride et al. (2021) [4] introduced the GRAVITAS framework, which uses model checking for verifying plans and goals in autonomous systems. GRAVITAS separates goal reasoning, plan verification, and execution. Our architecture emulates GRAVITAS' layered design. The Controller Agent is responsible for high-level planning (goal checks, execution triggers), while each AUV independently handles execution. This separation aligns with GRAVITAS' use of PAT to ensure logic consistency before information is sent to the actuators (in this case, through ROS). The controller agent verifies that exploration is complete before launching the rescue phase, just as GRAVITAS checks goal conditions before updating the plan. Furthermore, PAT-based verification in our system ensures that task transitions, such as moving from exploration to rescue, do not occur prematurely. This protects mission integrity and ensures the Rescuer AUV is only deployed once all relevant information is gathered. The 2025 assignment's emphasis on correctness through model checking and agent modularity is a direct extension of GRAVITAS' vision.

3 Proposed Approach

The proposed approach aims to implement a multi agent system for an autonomous aquatic search and rescue (SAR) implementation. This extends a prior single agent ROS + PAT implementation by introducing task specialisation and agent level coordination. The goal is to optimise the overall mission efficiency. This will be done by splitting the responsibilities into two distinct AUVs, which will be overseen by a third agent that manages task sequencing and inter-agent coordination.

The architecture consists of a search AUV that is responsible for environmental exploration and mapping, completing a sweep of the grid-based world to detect and record the location of all survivors using simulated sensors. The rescue AUV is then responsible for picking up all the survivors, using the information from the search agent, it relies on high level planning with PAT, to calculate the optimal path to retrieve all survivors. The third agent is the SAR Manager Agent, that is a supervisory controller, governing the systems temporal flow. It is in control of ensuring that the rescue agent is deployed only once the search agent has completed its task, to ensure no collisions between AUVs occur. This system is to be visualised and evaluated through a Gazebo simulation environment.

3.1 Environment Modelling and Sensors

The environment is modelled as a two-dimensional discrete grid which simulates an aquatic disaster zone populated with both survivors and hostiles. The survivors and hostiles are placed at random, non-overlapping locations, on each simulation run, which provide a diverse range of search and rescue scenarios. The simulation is rendered using Gazebo, where a survivor is represented by bowls, hostiles by boxes, and the AUVs by turtlebot3 models.

Each AUV has its own internal representation of the grid which is continuously updated via ROS-based sensor simulations. These sensors have directional range detection for North, East, South and West, to detect both hostiles and survivors. These sensor ranges can be modified to allow for experiments with varying sensor distances.

The search AUV uses these sensors to construct a map, which marks the explored tiles, detected enemies, and unreachable zones. As it traverses the grid, it updates the map and shares it with the SAR Manager agent. This agent then informs the rescue AUV to optimally navigate directly to survivor locations.

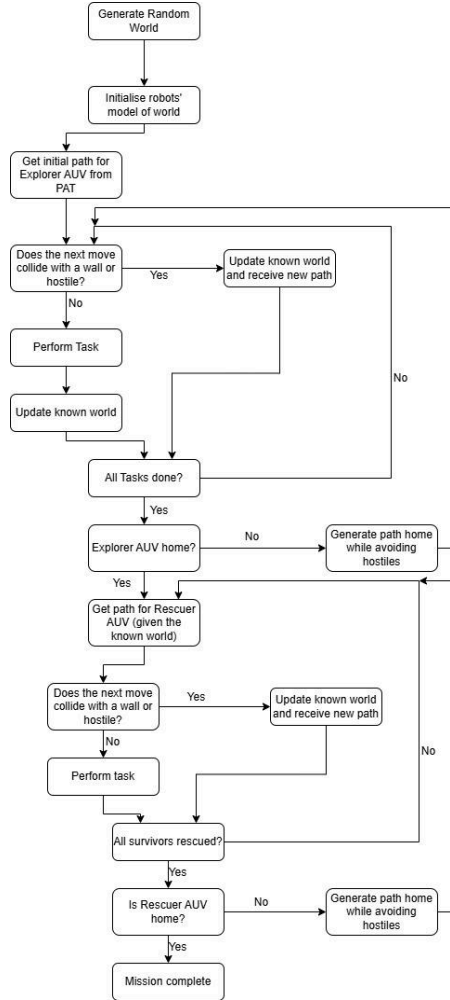


Figure 1: Overall workflow of the program

3.2 Control System Architecture

The control system logic for both AUVs is structured to contain 2 levels of high-level behaviours. For the explore AUV the two high-level behaviours it contains are surveying the environment for survivors and returning home, whereas the high-level behaviours for the rescue AUV are collecting survivors and returning home. Surveying the environment for survivors serves as the main behaviour of the search AUV as it attempts to find all of the survivors in the world and then return once they are all located. Collecting survivors serves as the main behaviour of the rescue AUV as it attempts to collect all of the already located survivors and then return them home.

Using this control structure enables both of the AUVs to perform their required task quickly and effectively, more details can be found within figure 2 and 3 below:

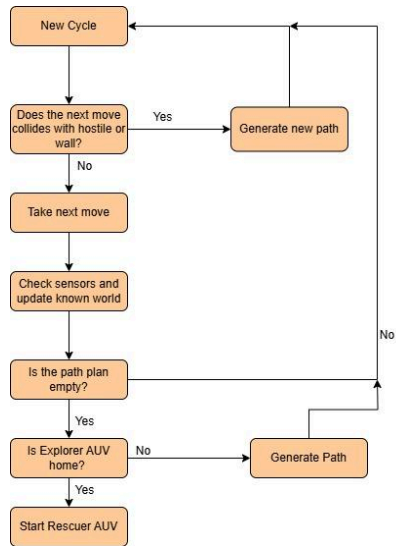


Figure 2: Submarine Control System Logic - Explorer AUV

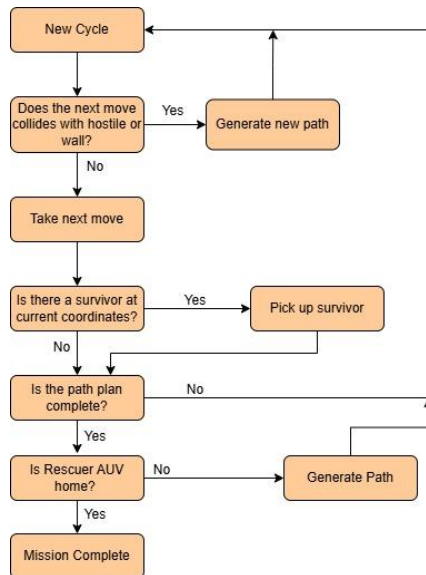


Figure 3: Submarine Control System Logic - Rescuer AUV

3.3 Integration of ROS and PAT

The proposed approach is designed to use PAT to calculate the planned path for each of the AUVs and then deliver the path in the form of a set of instructions for the AUVs to follow. ROS will then be responsible for executing the instructions given by PAT through its use of control logic, monitoring environment, environment variables and state for each AUV to help navigate and move around the AUVs. By integrating both PAT and ROS this approach aims to achieve a fast and efficient multi-agent that is able to more efficiently than the original single-agent system that contains one AUV.

4 Implementation and Experiments

4.1 Implementation Overview

The overview of this system is going to share a lot of similarities with the Implementation section from Todd et al. (2023) [2] due to the fact that this system is built off of their system. This means that the three tools mentioned in their report are also used in this system: ROS, PAT, and Gazebo. ROS is responsible for handling the main framework of the system including all functions and code, PAT is responsible for path planning, and Gazebo is responsible for simulating and visualizing the environment. Have these three tools be implemented together and communicate with each other to enable effective execution of the systems tasks and requirements:

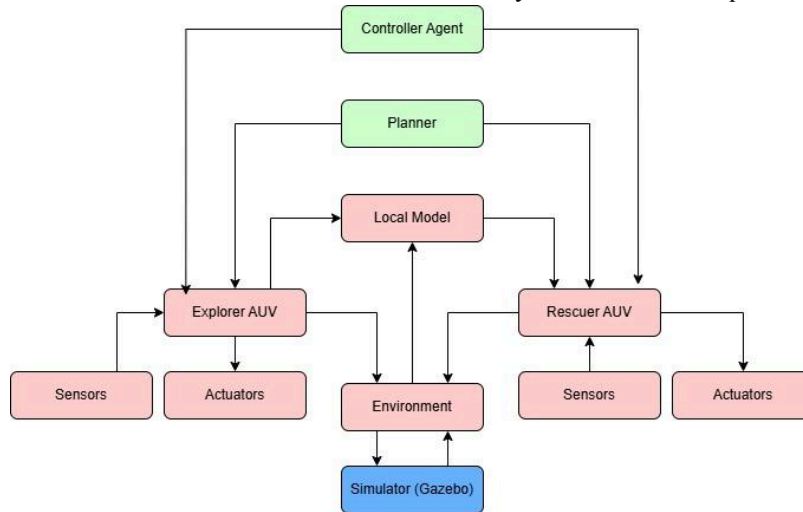


Figure 4: Overall System Flow

4.2 Implementation ROS (Robot Operating System):

ROS is programmed using C++ and for this system, there are 5 files that contain the important code for this system:

- search_rescue_search.cpp

- search_rescue_rescue.cpp
- search_rescue_manager.cpp
- update_grid.cpp
- communal_defines.cpp

The manager file and the update grid file are both files designed to be run with the terminal. Update grid is responsible for handling updating the 2D array and updating the placement of objects in Gazebo and the manager file is responsible for running both the SAR files when needed and as the search file will be run first it will be the file that randomly generates the world including placing the survivors, hostiles, and the two AUVs. Lastly communal defines are for communal definitions of values that will be used across the other ROS files.

4.3 Implementation PAT (Process Analysis Toolkit):

PAT is used to determine the next path that the robots should take by examining .csp files. ROS communicates through to a .csp file called “world.csp” which is updated to contain the current information for the 2D array and the position of the current AUV that is been moved. There are three navigation types that are programmed into the main path-finding files used by PAT, “path_search.csp” and “path_rescue.csp” for each respective AUV. These three goals are, explore environment, collect survivors, and return home. Exploring the environment is a goal only given to the search AUV and is triggered when the search AUV still has survivors to find or until the search AUV has explored every possible position it can reach on the grid. Collect survivors is a goal only given to the rescue AUV and triggers while there are still survivors on the grid to collect and while the capacity of the AUV is not exceeded. The last goal is return home which is used by both the search and rescue AUV to return to their home points, starting points, with the only difference between the return home goal for each respective AUV being the home location each AUV is returned to. Below are more detailed specifications for each goal:

goalAreaChecked:

- Explore all unvisited positions to try and find survivors
- Goal is only used by search AUV
- Definition, (&& i: {0..Rows-1} @(&& j: {0..Cols-1} @(world[i][j] == Visited || world[i][j] == Hostile || world[i][j] == Survivor)) && home);
- Uses UnvisitedMove() behaviour to prioritize taking moves that pass through unvisited locations

noSurvivors:

- Collect all survivors on the map and if capacity is full then return home to deliver survivors
- The check for finding all survivors is handle within ROS
- Goal is only used by rescue AUV
- Definition, (($\&\& i: \{0..Rows-1\} @ (\&\& j: \{0..Cols-1\} @ (world[i][j] \neq Survivor))$ $\&\& (onBoard \neq maxCapacity \parallel home)) \parallel (onBoard == maxCapacity \&\& home));$
- Uses UnvisitedMove() behaviour

Home:

- Return to start point of the AUV
- Goal is used for both search and rescue AUV
- Definition (search sub): ($xpos == SUB_HOME_X \&\& ypos == SUB_HOME_Y$);
- Definition (rescue sub): ($xpos == SUB_HOME_X+1 \&\& ypos == SUB_HOME_Y$);
- Use VisitedMove() behaviour to prioritize taking moves that pass through visited locations

4.4 Implementation Gazebo:

ROS is able to use the updateGrid service to interact with Gazebo to have it visually represent the locations of the AUVs hostiles and survivors found within the 2D array used within the ROS files. Gazebo has been set up to ensure that the movement of both AUVs are displayed correctly in real time by constantly updating Gazebo with the current array data.

4.5 Testcases Experiments (randomly generated test case):

These testcases were all ran on the same device to ensure that hardware differences were not having an effect on the results of the testing. The specifications of the device are:

OS – Windows 11

CPU – 11th Gen Intel® Core™ i7

GPU – NVIDIA GeForce RTX 3050 Ti Laptop

RAM – 32GB

To run your own experiments within the terminal first the files need to be compiled into ROS packages by using “catkin_make”. Note, if you have previously used a ROS package called “assignment_3” or used a folder called “[3806ict_assignment_3](#)” then “catkin_make clean” is needed to ensure that the previous package does not cause an error. An error is also known when after using “catkin_make clean” the first “catkin_make” will cause an error, however repeating the command again will fix the issue and have the packages be made correctly. Then within the different terminals within the “catkin_ws” run these three following commands within the order they are stated:

1. `roslaunch assignment_3 launch_world.launch`
2. `roslaunch assignment_3 update_grid`
3. `roslaunch assignment_3 search_rescue_manager`

Below are the test cases that have been completed by running the system normally:

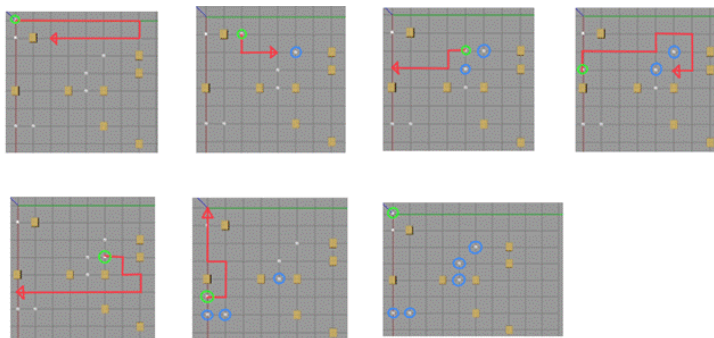


Figure 5: Randomly generated test case search AUV's path (red line – current path, green circle – search AUV, blue circles – discovered survivors)

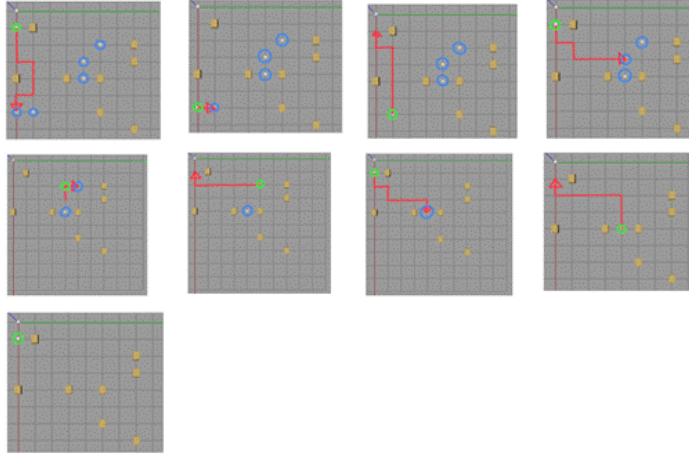


Figure 6: Randomly generated test case rescue AUV's path (red line – current path, green circle – rescue AUV, blue circles – discovered survivors)

4.6 Testcases Experiments (pre-made test cases):

Below are additional test cases completed using a pre-made world for the robot to navigate. The reason for conducting these tests was to compare the new multi-agent system to the original system from Todd et al. (2023) [2] to see if this new system is actually faster. Having these test cases be pre-made and shared between the two systems means that the results will not be affected by how each area is generated for each system, e.g., the original systems gets an easy area to navigate and the multi-agent system gets a hard area to navigate. These tests were completed on the same device/ hardware that was listed in the previous section:

Pre-Made Test Case	Original System	Multi-Agent System
1	40.3 seconds	58.3 seconds
2	34.9 seconds	51.8 seconds
3	36.5 seconds	53.6 seconds
4	32.7 seconds	1 minute 8.3 seconds
5	55.7 seconds	49.2 seconds

6	45.2 seconds	53.3 seconds
7	50.3 seconds	1 minute 9 seconds
8	35.1 seconds	58.4 seconds
9	39.8 seconds	45.1 seconds
10	44.2 seconds	48.6 seconds
Average	41.47 seconds	55.56 seconds

Table 1: time for each system to complete each pre-made test case

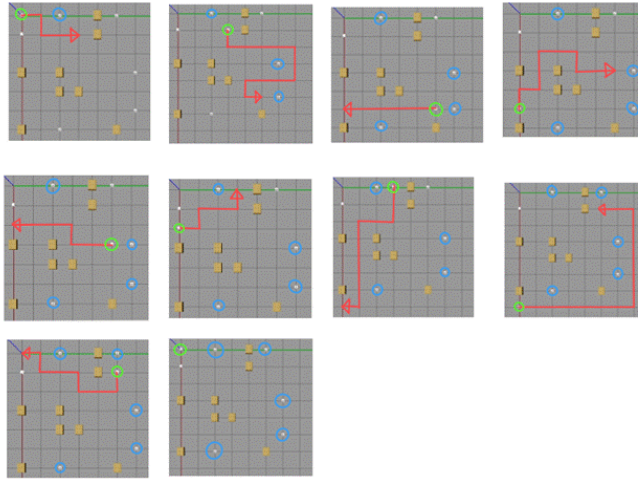


Figure 6: Pre-made test case 4 multi-agent system search AUV's path (red line – current path, green circle – search AUV, blue circles – discovered survivors)

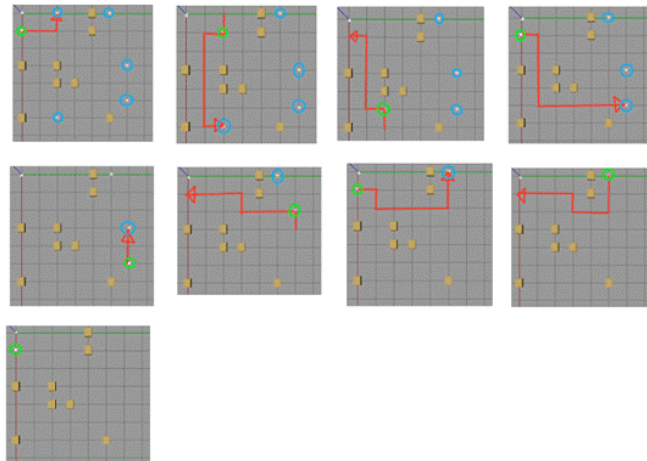


Figure 7: Pre-made test case 4 multi-agent system rescue AUV's path (red line – current path, green circle – rescue AUV, blue circles – discovered survivors)

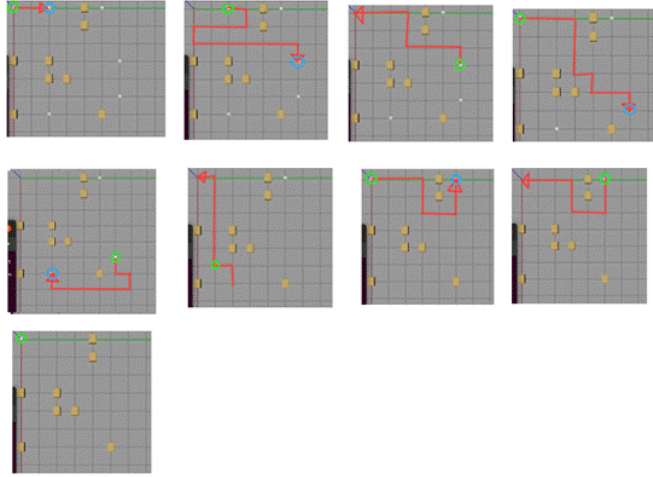


Figure 8: Pre-made test case 4 original system AUV's path (red line – current path, green circle – AUV, blue circles – discovered survivors)

4.7 Experiments Results:

The objective of the first test case, a randomly generated test case, was to demonstrate how the two agents function together under normal circumstances without having a pre-made map that they are certain to complete. The paths demonstrated within fig 5 and 6 show how these agents use BFS to navigate to either find the survivors or rescue them and given that this test case was successful it shows that the two-agent system is successful in its goal of rescuing all survivors. Replicating this test case will not always lead to success as there are limitations with how the multi-agent system functions, this will be addressed in the discussion section. The pre-made test cases were made to examine if this multi-agent system is a faster system than the original this system was built from. Examining the times recorded for the systems to complete the pre-made test cases 1-10 in Table 1 it is clear that the original system is noticeably faster than the two-agent system. The only case where this isn't the case is test case 5 and as this seems to be an outlier it is assumed that the original system is the faster system. Given the paths shown within figures 6,7, and 8 the total distance/ length of the AUV takes is noticeably shorter meaning the total time executing and calculating all of the moves will be longer for the two-agent system when compared to the original system. As an additional note, some of the screenshots in the figures show the AUV in a position just after the start of the path and this is due to the time taken to delete the survivor object and then move the AUV onto the spot to collect them being longer than the time it takes to process and move the AUV to the next position.

5 Discussion/Conclusion

5.1 Previous Limitations:

Given that the multi-agent system is created off of the code provided from Todd et al. (2023) [2] system, all of the limitation mentioned within their report carry over to this system as this system was purely designed add the implementation of a two agent system and not to fix any of the limitations of the already existing code:

- **Computationally Expensive Search Algorithms:**
 - Computational complexity associated with the search algorithms used for path planning in PAT
 - State space becomes excessively large, the BFS algorithm expands exponentially.
 - Major issues include, exceeding memory capacity and prolonged search times
 - Solution - alter the search function used, BFS, to test if another search algorithm will result in a reduced complexity
- **Control System for Realistic Movement:**
 - grid based movement and teleporting is not realistic
 - A more indepth movement system should be used
 - Solution - Adapt a more realistic movement system for the AUV, or even adapt a real life AUV movement system for use in the system
- **Handling Energy Consumption:**
 - AUV currently operates with an unlimited amount of energy
 - Energy consumption and management is an important of real world SAR missions and should be accounted for in this system
 - Solution - study real world energy usages of AUVs to design and implement a realistic system for energy consumption
- **Isolated survivors:**
 - survivors can be located in areas that cannot be accessed by the AUV meaning that they cannot be rescued
 - this also extends to the AUV been boxed into its starting location by hostiles
 - solution - include an extra level of decision making to enable either flagging of their location, allow additional action to enable the AUV to reach them, or make the world generation not allow impossible to reach survivors to be placed

5.2 New Limitations:

These are the new limitations that now exist within the system when it was updated to become a multi-agent system:

- Additional spawn issues due to second AUV and search AUV's inability to collect survivors:

- due to the addition of a second AUV there are now more possibilities for the AUVs to be blocked in as currently the second AUV spawns below the first
- if a survivor is blocked by another survivor and collecting the first survivor is the only way to see the second then the search AUV will be unable to spot the survivor and in-turn will be unable to locate all survivors
- solutions - alter the spawn of the second AUV so it spawns in another corner of the world so it does not block the search AUV from exploring the world to locate all the survivors, and for survivors blocking survivors, alter the sensors to be able to see further and identify objects further away so the second survivor can be spotted and marked on the world
- Additional time used search the map searching for survivors:
 - In a rescue situation time is often a limited resource and having the search AUV spend a large amount of time total complete just looking for survivors and rescuing them is not ideal in a real world situation
 - solution - having the search AUV still pick up the survivors it encounters but still remain to explore even when it's max capacity is reached would allow the some of the survivors to be quickly saved, this would in theory also reduce the overall time for the rescues completion

5.3 Evaluation:

The multi-agent system that was implemented does complete what it was designed to do in terms of it having two AUV work together with one AUV assigned to searching the area for survivors while the other is designed for rescuing the survivors. However in terms of being more efficient than the original system this goal was not met as the multi-agent system is slow to complete pre-set rescues when compared to the original system. There are potential improvements that can be made to help speed up the completion of the multi-agent system, however with the current limitations of this system it cannot be recommended over the original system.

5.4 Potential Improvements

One major factor contributing to the longer execution time of the multi agent system is its strictly sequential agent execution. The rescue AUV is not allowed to begin its task until the search AUV has completely swept the environment and returned home. This results in a period of inactivity, where the rescue agent is idle, when it could be deployed to start picking up survivors. A Potential improvement to combat this issue is deploying the rescue agent, while the search agent is still completing its mission, this would significantly improve execution time, however this could also increase the chance of a collision between the AUVs. Another improvement could be for the search agent to pick up survivors that lie in its direct path, this would allow for the rescue agent to have a shorter path, ultimately improving execution time.

5.5 Conclusion:

In conclusion this report presents a multi-agent that is designed to conduct SAR missions with hostile areas in an attempt to rescue survivors. It uses ROS as a robust platform and PAT as an intelligent decision-making framework to control and guide the AUVs as they move around the world to complete their SAR mission by locating, collecting, and returning home with all the survivors. This current system is most certainly functional in these test case scenarios, however, if this system is to be implemented within the real world then its limitations must be addressed to ensure that the survivors in the SAV situation are able to be rescued safely and quickly.

6 References

1. Maxey, L.: Bluefin-21 dives deep in search of Malaysia Flight 370. Engineering.com (2014), <https://www.engineering.com/bluefin-21-dives-deep-in-search-of-malaysia-flight-370/>, last accessed 2025/05/26.
2. Tood, C., Callam, H., James, H., Daniel, J.: Autonomous Search and Rescue: Griffith University, Southport QLD, Australia (2023).
3. Venkatesan, S.: AUV for Search & Rescue at sea - an innovative approach. 2016 IEEE/OES Autonomous Underwater Vehicles (AUV), 1-9, (2016).
4. Bride, H., Dong, J. S., Green, R., Hou, Z., Mahony, B., Oxenham, M.: GRAVITAS: A Model Checking Based Planning and Goal Reasoning Framework for Autonomous Systems. Engineering Applications of Artificial Intelligence 97 (2019)

7 Execution Details

The following link is for the public GitHub repository that contains all of the code for the multi-agent system. This repository contains all PAT .csp files, .cpp files, and all files needed to make “catkin_make” create the ROS packages, e.g. cmake file:

- <https://github.com/CallumABrown/Assignment-2-3806ICT-group-6>